HDOC DAY – 53

Algorithm

1. Start.
2. Read the size n.
3. Declare the variable-length integer array nums[n].
4. Read all elements into nums[].
5. Pass the address of nums[] and n to product_except_self().
6. Inside the function:
    - Declare answer[n].
    - Initialize prefix = 1.
    - Traverse from left to right:
        - Store prefix in answer[i].
        - Multiply prefix by nums[i].
    - Initialize suffix = 1.
    - Traverse from right to left:
        - Multiply answer[i] by suffix.
        - Multiply suffix by nums[i].
7. Display answer[], separated by commas.
8. Stop.

C PROGRAM:

```
#include <stdio.h>


void product_except_self(int *nums, int n)
{
   int answer[n];
   int prefix = 1, suffix = 1;
```

```
    // Store product of all elements to the LEFT of nums[i]
    for (int i = 0; i < n; i++)
    {
        answer[i] = prefix;
        prefix = prefix * (*(nums + i));
    }

    // Multiply by product of all elements to the RIGHT of nums[i]
    for (int i = n - 1; i >= 0; i--)
    {
        answer[i] = answer[i] * suffix;
        suffix = suffix * (*(nums + i));
    }

    // Display answer array
    for (int i = 0; i < n; i++)
    {
        printf("%d", answer[i]);

        if (i < n - 1)
            printf(", ");
    }

    printf("\n");
}

int main()
{
```

```
    int n;

    printf("Enter the size of array: ");
    scanf("%d", &n);

    int nums[n];

    printf("Enter %d elements: ", n);

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &nums[i]);
    }

    // Passing address of array
    product_except_self(nums, n);

    return 0;
}
```

TEST CASE TABLE:

Test Case	Nums []	Expected Output
1	1 2 3 4	24, 12, 8, 6
2	-1 1 0 -3 3	0, 0, 9, 0, 0
3	2 3 4	12, 8, 6
4	1 1 1 1	1, 1, 1, 1
5	2 0 3	0, 6, 0